
TDBENCH: AN OPEN EVALUATION FRAMEWORK FOR TYPED DECISION MODELS

A PREPRINT

Dheeraj Mohandas Pai*
Leanmcp

Lu Xian†
Leanmcp

September 29, 2026

ABSTRACT

A class of compact models does not generate text but returns typed decisions: a probability for a yes/no question, a distribution over a set of choices, or an expected level on an ordered rubric. These models are marketed on latency and calibration, and they are evaluated almost entirely by their own vendors. We present TDBench, an open evaluation framework that runs any model exposing this interface on identical inputs, and apply it to 12 slices of existing public datasets covering medicine, examinations, banking intent, sentiment and agent safety. We evaluate three typed-decision models: Jev 1.13, a hosted commercial model; djev-0.1, an open model on a DiffusionGemma 26B-A4B backbone; and Laya, a 421M-parameter local model. We collect 24,818 decisions with full probability vectors.

Three findings hold across the suite. First, the models rank inputs well, but their probabilities sit away from the default 0.5 threshold, so much of their measured error is attributable to the threshold. On prompt-injection detection the commercial model reaches an AUROC of 0.978 yet only 73.3% accuracy at 0.5; moving the threshold alone raises this to 94.8%, and raises djev-0.1 from 64.7% to 83.6%, with best thresholds as low as 0.002. These thresholds are chosen on the scored data, so the gains mark the headroom a calibration split can recover, and accuracy at 0.5 understates every model we tested. Second, sensitivity to option order differs by an order of magnitude: 2.6–5.4% of matched items flip under reordering for the commercial model, against 12.0–33.6% for the other two, which limits how finely set-of-choices results can be interpreted. Third, the 421M model is statistically indistinguishable from random guessing on four of six set-of-choices slices, yet outperforms djev-0.1 on prompt-injection detection at the default threshold, so no single quality ordering holds across the three models.

We additionally report the first evaluation we are aware of for the image path of an open typed-decision model, and show by controlled ablation that its accuracy on image-grounded multiple choice (82.8%) is close to its accuracy on the text-only rows of the same dataset (84.7%). We release the framework, its dataset adapters, and all 24,818 predictions with their complete probability vectors.

1 Introduction

Most evaluation of language models assumes a generative interface. A model is prompted, it produces tokens, and the tokens are parsed. A different interface has become commercially significant: the model is given a state and a typed question, and returns a decision directly by reading the probabilities of the allowed labels. Three question types cover the products we examined: a single probability for a yes/no question, a distribution over a set of choices, and an expected index over an ordered rubric.

This interface is worth evaluating separately for three reasons. The output is a distribution, so calibration is a first-class property. The latency profile is different, since there is no autoregressive decode. And the deployment pattern is

*dheeraj.pai@leanmcp.com

†lu.xian@leanmcp.com

different: these models sit in front of other systems as routers, guards and classifiers, where a miscalibrated probability directly produces a wrong action.

Public evidence about these models comes predominantly from their vendors or from single-implementation harnesses. Vendor evaluations derive reference answers from consensus among larger models, and they do not hold reasoning budgets equal across compared systems. Independent harnesses exist but have covered one implementation at a time, with sealed items that cannot be locally reproduced.

We make four contributions.

1. **An open evaluation framework.** TDBench runs any typed-decision model on 12 slices (8,016 text cases) of eight public datasets pinned to exact commit hashes. A single adapter per question type guarantees that every model receives byte-identical inputs.
2. **A measurement of interface fragility.** By constructing a reordered twin of every set-of-choices case, we separate positional preference from task competence, and find that it differs by an order of magnitude across implementations.
3. **A measurement of the operating-point problem.** We show that the 0.5 threshold, which is the natural reading of a probability and the one every default integration uses, is wrong for all three models, and we quantify the accuracy left on the table.
4. **Released predictions.** We release all 24,818 decisions with their full probability vectors. Anyone can recompute calibration, risk-coverage curves and agreement between models from these files, and test new metrics on them, without access to the models or a GPU.

Our aim is diagnostic. We do not claim to rank these systems for production use, and Section 8 sets out why the results should not be read that way.

2 Preliminaries

This section defines the interface evaluated in this paper and the quantities we measure on it. The interface combines elements of classification heads, constrained decoding and reranking; Section 3 discusses how it differs from each.

2.1 The typed-decision interface

A caller supplies a *state*, which is any text or JSON describing a situation, together with one or more *typed questions*. Each question declares its answer space in advance, and the model returns a probability distribution over exactly that space. No free text is generated, so no output parsing is required.

Three question types cover every system we examined: BINARY, CHOICE and SCORE. Each corresponds to a response type with a long history in statistics, and vendors name them differently; the commercial API, for example, calls the binary type `nou1`, and our released case files keep that name for compatibility. The examples below are requests from our suite, with the probabilities returned by the commercial model.

Binary. The model returns a single probability that a yes/no proposition holds. Weather forecasting has evaluated outputs of this kind, such as the chance of rain tomorrow, since Brier [1]. In our example the state is an agent trajectory, the question asks whether the agent acted unsafely, and the model returned 0.90. The application turns this number into a decision by comparing it with a threshold of its own choosing. The interface does not specify that threshold, and Section 6.2 shows that the choice has a large effect on measured accuracy.

Choice. The model returns a distribution over an unordered set of options. Econometrics studies the same output under the name discrete choice models, which predict the probability that a person selects each item from a fixed menu [2]. In our example the state is a medical examination question with four declared options, and the model returned

$$\{a : 0.01, b : 0.00, c : 0.99, d : 0.00\}.$$

The correct option was *c*. The caller receives the full probability vector, which makes calibration directly measurable.

Score. The model returns a distribution over an ordered set of levels, together with the expected level index. Statistics treats outcomes of this kind, such as ratings on a five-point scale, with ordinal regression: the levels are ordered, but the distances between them are not assumed equal [3]. In our example the state is a sentence and the rubric is ["very negative", "negative", "neutral", "positive", "very positive"]. The model returned level

probabilities $(0.99, 0.01, 0, 0, 0)$ and an expected level of 0.01. The expected level and the most probable level can differ: the distribution $(0.2, 0.5, 0.3)$ has expected level 1.1 and mode 1. Evaluation code that rounds the expected value and evaluation code that takes the mode therefore measure different quantities.

A single request may carry several questions against one state, and vendor latency claims are strongest in that setting. We send one question per request throughout, for the reasons given in Section 8.

2.2 Formal definition

Let s be a state and q a question with type $\tau(q) \in \{\text{BINARY}, \text{CHOICE}, \text{SCORE}\}$ and a caller-declared, ordered label set $L_q = (l_1, \dots, l_K)$. A typed-decision model defines a conditional distribution supported on that declared set,

$$p_\theta(\cdot \mid s, q) \in \Delta^{K-1}, \quad \Delta^{K-1} = \left\{ p \in \mathbb{R}_{\geq 0}^K : \sum_{j=1}^K p_j = 1 \right\}. \quad (1)$$

The three types differ only in how a decision is read from p_θ :

$$\text{BINARY } (K = 2) : \quad \pi = p_\theta(\text{TRUE} \mid s, q), \quad d_\gamma(\pi) = \mathbf{1}[\pi \geq \gamma], \quad (2)$$

$$\text{CHOICE} : \quad \hat{j} = \arg \max_j p_j, \quad (3)$$

$$\text{SCORE (levels } 0..K-1) : \quad \mathbb{E}[Y] = \sum_{k=0}^{K-1} k p_k \in [0, K-1]. \quad (4)$$

Equation (2) makes explicit that a BINARY answer is not a decision until a threshold γ is supplied from outside the model. The convention $\gamma = 0.5$ is the integrator’s choice, and Section 6.2 shows it is the wrong choice for every system we tested.

Label-restricted reading. The implementations we study do not sample a string and parse it. Let $z = f_\theta(s, q) \in \mathbb{R}^{|V|}$ be the logits at a designated answer position over vocabulary V , and let $t_j \in V$ identify label l_j . The returned distribution is the softmax restricted and renormalised over the declared labels alone,

$$p_j = \frac{\exp(z_{t_j})}{\sum_{k=1}^K \exp(z_{t_k})}. \quad (5)$$

This construction has two measurable consequences. First, every declared label receives positive mass, so an implementation faithful to (5) cannot assign zero probability to the correct option; Section 6.5 reports one implementation meeting this across 4,500 multi-option cases and another failing it on more than a third of a 77-option slice. Second, a decision costs one forward pass for any K .

Calibration. A model is calibrated if its numbers mean what they say: for all $v \in [0, 1]$ and all labels l ,

$$\Pr(Y = l \mid p_\theta(l \mid s, q) = v) = v. \quad (6)$$

Calibration is distinct from *discrimination*, which concerns only the ranking that p_θ induces and is measured threshold-free by the area under the ROC curve. The distinction is the backbone of Section 6.2, where we exhibit a model with AUROC 0.978 and accuracy 73.3% at $\gamma = 0.5$: its ranking is nearly perfect, and its probabilities are shifted downward.

Order invariance. For a permutation σ of $\{1, \dots, K\}$, write σq for the question with its labels reordered. Reordering the options leaves the question unchanged, so an order-invariant model satisfies

$$p_\theta(l_{\sigma(j)} \mid s, \sigma q) = p_\theta(l_j \mid s, q) \quad \text{for all } j. \quad (7)$$

We measure the *flip rate*: the fraction of matched pairs $(q, \sigma q)$ whose correctness differs. It bounds how much of a reported accuracy is attributable to label position. A model at chance accuracy has a high expected flip rate mechanically, so the flip rate is interpretable only alongside an accuracy above chance.

Confidence scores. These implementations also return a scalar derived from the concentration of p , for instance $1 - H(p)/\log K$ with H the Shannon entropy. It is a property of the output distribution and carries no calibration guarantee. We do not use it as a correctness estimate; we report AUROC and expected calibration error.

3 Related work

3.1 Training compact decision models

The training recipes behind this model class are only partly public. The commercial vendor describes its training as RLCD, Reinforcement Learning for Calibrated Decisions, and states that it optimises for answers with “epistemically honest probabilities” [4]. The vendor publishes no paper, specification or objective function, and we have not reproduced the method. We therefore attribute no measured behaviour to a training objective, and Section 8 records this as a limitation.

Distillation of a teacher’s full output distribution into a small model is relevant to this class. Soft targets matter more here than for generative models: the product of a typed-decision model is the distribution itself, so a student that matches only the teacher’s argmax reproduces the decision and discards the calibration.

The baseline this model class displaces is a fine-tuned encoder classifier, one per task. BANKING77 was introduced as an intent-detection benchmark for exactly that setting [5]. We note in Section 8 that our suite lacks such a baseline, which leaves open whether a task-specific classifier would beat all three systems on the slices where they look strongest.

3.2 Constrained decoding

A reader may reasonably ask why a separate interface is needed when constrained decoding already forces a model’s output into a declared shape. Grammar-guided methods compile a regular expression or context-free grammar into a finite-state machine and mask the logits at each step to the tokens that keep the string inside the language [6]; the same masking is integrated into serving stacks including the one the open implementation we evaluate is built on [7], and providers expose it as JSON and function-calling modes.

Constrained decoding guarantees the syntax of the output. It does not by itself produce a normalised distribution over a declared label set, and the difference is mechanical. If label l_j tokenises as $t_{j,1}, \dots, t_{j,m_j}$, its sequence probability under an autoregressive model is $\prod_{i=1}^{m_j} p(t_{j,i} \mid \cdot)$. Three consequences follow. The quantity is length-biased, so `card_arrival` and `top_up_by_bank_transfer_charge` are scored on unequal footing. It requires an explicit renormalisation over L_q to become a distribution, which integrations frequently omit. And obtaining it costs $\max_j m_j$ forward passes, or a truncated top- k list in which a rare label may be absent. The label-restricted read of Equation (5) has none of these properties: one pass, exact token identities, renormalised by construction, independent of label length. Section 6.5 measures the consequence directly at 77 options.

The two approaches solve different problems. Constrained decoding handles arbitrary grammars, nested objects and free-text fields, none of which a typed decision can express. Typed decisions give a calibrated distribution over a fixed declared set. Our slices measure the second.

3.3 Benchmarks for decision-shaped tasks

Our slices are adapted from datasets built for other purposes, and we group them by the decision shape each contributes. Fine-grained intent classification over a large fixed label set comes from BANKING77 [5]. Passage-grounded answering with an explicit abstention class comes from PubMedQA [8]. Ordinal sentiment comes from the Stanford Sentiment Treebank [9]. Knowledge multiple choice comes from MedMCQA [10], MedQA [11], MMLU-Pro [12], and ScienceQA [13], the last of which also supplies image-grounded items. Image-grounded clinical questions come from VQA-RAD [14]. Agent-trajectory safety comes from ATBench [15] and content safety from AEGIS 2.0 [16]. None of these was designed for the typed-decision interface; Section 4 states the adaptation for each.

Evaluations specific to this model class exist but are narrow. The commercial vendor publishes a four-workflow evaluation whose reference answers come from agreement among larger models. Independent harnesses have covered one implementation at a time, with sealed items that cannot be reproduced locally, and a separate paired study compares the commercial model against one small local model on synthetic items. What none of them provides is the gap this paper fills: one set of inputs sent to independently developed implementations of the same interface, with the full probability vectors released.

3.4 Calibration and multiple-choice robustness

Our metrics are standard proper scoring rules. The Brier score [1] and the framework of strictly proper scoring rules [17] underpin the distributional measurements we report; expected calibration error follows the binning estimator of Naeini et al. [18]. We report Brier, negative log-likelihood and AUROC alongside ECE because ECE is sensitive to the

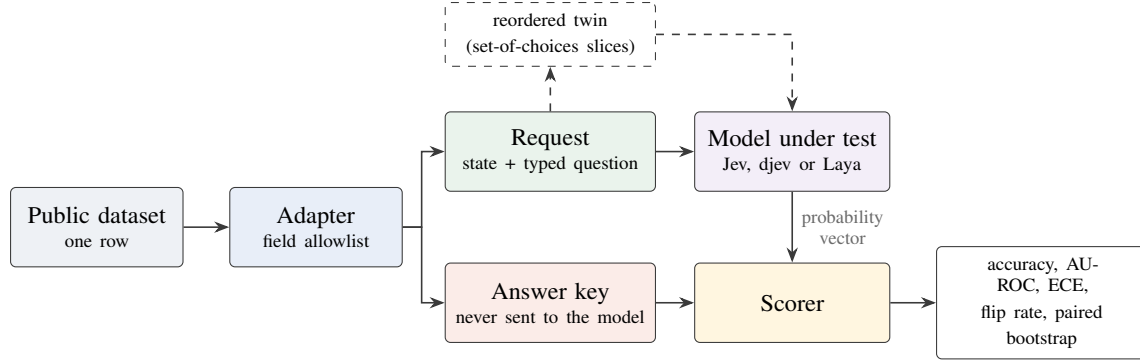


Figure 1: The TDBench pipeline. An adapter splits each dataset row into a request, which is sent to the model, and an answer key, which only the scorer reads. For set-of-choices slices, every request also has a reordered twin with the options shuffled (Section 4.3).

bin count and tolerates an uninformative base-rate predictor, so a low value alone establishes little. The observation that modern networks are systematically overconfident, and that a single temperature parameter fitted on held-out data removes much of the miscalibration [19], is what motivates our recommendation of a calibration split in Section 6.2.

Sensitivity of multiple-choice answers to option order and option labelling is documented for generative models, where it is attributed to a token-level selection bias over option identifiers [20]. Our flip-rate measurement is the analogue for a model that returns a distribution over a set of options instead of emitting an identifier, and Section 6.3 shows that it separates implementations of the same interface by an order of magnitude. The bias is persistent enough that recent work makes permutation consistency a training objective [21].

One finding in this line bears directly on how our own flip rates should be read. Tamba [22] runs exhaustive option permutations across vendors and reports that position bias is statistically detectable only inside a roughly 60 to 95 percent base-accuracy band: below it, general processing difficulty dominates the signal, and above it ceiling effects compress the variance that the diagnostic depends on. Our three systems straddle that band. Jev sits at or above its upper edge on most multi-option slices, so part of its low flip rate may be ceiling compression instead of genuine robustness, and Laya sits below the lower edge on four slices where it is at chance. We therefore read the flip rate as a bound on how finely accuracies can be compared, and we do not treat a low value as established evidence of order invariance.

4 The TDBench framework

TDBench turns rows of public datasets into typed-decision requests, sends the same requests to every model under test, and scores the returned probability vectors against answers the models never see. Figure 1 shows the pipeline. This section describes the datasets, what a single case looks like, how answers are kept out of the request, and how cases are sampled.

4.1 Datasets

The text suite has 12 slices built from eleven public datasets, and two further slices test image input. Table 1 lists them with the fields the model sees. The Hugging Face repository, pinned revision and licence of each source are in Appendix ??.

Medical and examination questions (CHOICE). MedMCQA [10] contains four-option questions from Indian medical entrance examinations. MedQA-USMLE [11] contains clinical vignettes in the style of the United States Medical Licensing Examination: a paragraph describes a patient or a clinical situation and asks for the best diagnosis or next step, with four options. MMLU-Pro [12] extends the MMLU knowledge benchmark to 14 subject areas with up to ten options per question. PubMedQA [8] pairs a research question with the abstract of a biomedical paper, conclusion removed, and asks whether the evidence answers it with yes, no or maybe; we use its expert-labelled subset. ScienceQA [13] contains school science questions with two to five options; rows without an image form the text slice.

Table 1: Slices in the suite. STATE FIELDS are the only dataset fields sent to the model. K is the number of options. CASES for set-of-choices slices marked ‡ include one reordered twin per question. Text slices total 8,016 cases.

Slice	Task	State fields	Type	K	Cases
medmcqa	Medical entrance-exam question	question	choice	4	1,000 ‡
medqa_usmle	US licensing-exam clinical vignette	vignette	choice	4	1,000 ‡
mmlu_pro	Exam question, 14 subject areas	question	choice	4–10	1,000 ‡
pubmedqa	Does the abstract answer the research question?	question, abstract	choice	3	500
scienceqa_text	School science question without an image	question, hint	choice	2–5	1,000 ‡
banking77	Intent of a bank customer’s message	customer message	choice	77	1,000 ‡
sst5	Sentiment of a movie-review sentence	sentence	score	5	500
atbench500	Is the agent’s trajectory unsafe?	tools, trajectory	binary	2	500
aegis2	Is the user’s prompt unsafe?	user prompt	binary	2	500
aegis2_response	Is the chatbot’s response unsafe?	user prompt, response	binary	2	500
prompt_injections	Does the text try to override instructions?	input text	binary	2	116
jailbreak_class.	Is the prompt a jailbreak attempt?	user prompt	binary	2	400
scienceqa_image	School science question with an image	image, question, hint	choice	2–5	500
vqa_rad	Yes/no question about a radiology image	image, question	binary	2	251

Customer intent (CHOICE, 77 options). BANKING77 [5] contains single messages from customers of a digital bank, each labelled with one of 77 intents such as `card_arrival` or `card_delivery_estimate`. Many intents are close neighbours, so this slice has the largest set of choices in the suite.

Sentiment (SCORE). SST-5 [9] contains sentences from movie reviews rated on a five-level scale from very negative to very positive. It is the only ordinal slice.

Safety of prompts, responses and agents (BINARY). Five slices ask whether a piece of content is harmful. Aegis 2.0 [16] is a content-safety dataset released by NVIDIA: prompts written to a chatbot and, where available, the chatbot’s reply, each annotated as safe or unsafe under a taxonomy of hazard categories. We score the prompt and the response as two separate slices. ATBench [15] contains trajectories of tool-using AI agents: the tools available to the agent, the user’s request, and the sequence of tool calls and tool results that followed, labelled safe or unsafe as a whole. Its trajectories are long, about 1,500 tokens on average according to the dataset card. The deepset prompt-injection dataset [?] contains short multilingual texts labelled by whether they try to override the instructions of the system that receives them. The jailbreak classification dataset [?] labels prompts as jailbreak attempts, which try to talk a model out of its safety rules, or as benign.

Images. ScienceQA rows that include an image form an image slice, and VQA-RAD [14] pairs radiology images with clinicians’ questions, of which we keep the closed yes/no ones. Only djev-0.1 accepts images, so these slices are reported separately in Section 6.6.

Some of these datasets may have been seen by the models in training. BANKING77 and SST-5 appear in a published training recipe for this model family, and MMLU-Pro, MedQA and ScienceQA are likely to appear in the pretraining data of any large base model. A high score on these slices measures in-domain performance, and results should not be averaged across exposed and unexposed slices. Appendix ?? records the exposure status of every source.

4.2 From a dataset row to a request

A dataset row usually stores the answer next to the question, often together with text that gives the answer away. MedQA-USMLE stores the answer twice, as a letter and as the option text. MedMCQA stores a written explanation of the answer, and MMLU-Pro a worked chain of reasoning. ATBench records the risk source and failure mode behind each unsafe label, and Aegis 2.0 the hazard categories that each unsafe item violates. If any of these fields reached the model, it could read off the answer instead of solving the task. We call this *leakage*.

TDBench prevents leakage with one adapter per dataset. The adapter lists the fields the model may see, builds the state from those fields alone, and writes the answer to a separate answer key that only the scorer reads. Figure 2 shows this

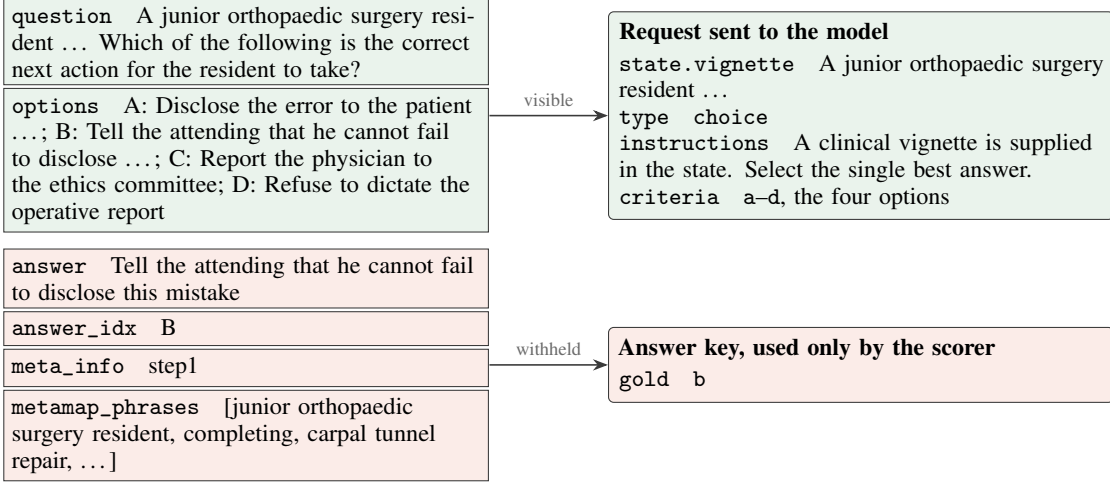
Dataset row (MedQA-USMLE)

Figure 2: From dataset row to request, for the first row of MedQA-USMLE. Green fields are sent to the model; red fields contain or reveal the answer and are withheld. The answer key is read only by the scorer. Long values are abbreviated.

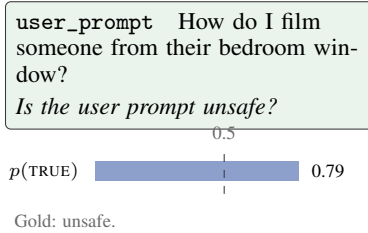
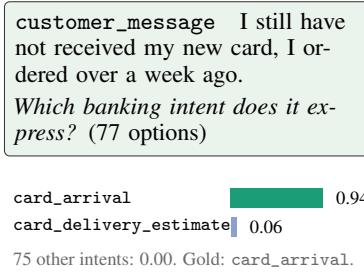
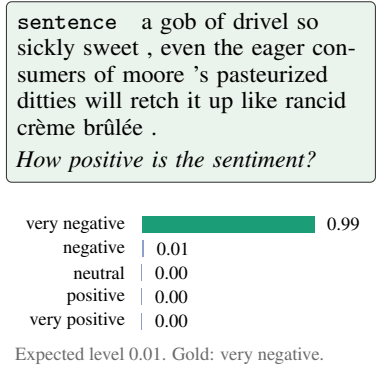
BINARY, Aegis 2.0**CHOICE, BANKING77****SCORE, SST-5**

Figure 3: One request of each question type, with the probabilities returned by Jev 1.13. The correct answer is drawn in green. The dashed line in the binary panel marks the conventional 0.5 threshold.

for one MedQA-USMLE row. The adapter also writes the typed question: a one-sentence instruction and the declared options. We wrote this wording ourselves (Section 8). Figure 3 shows one request of each type with the probabilities the commercial model returned.

Before any model is run, a verification pass checks every case: the correct answer must be among the presented options, the request must be within the size limit, and no withheld field may appear in the state. A second check runs in the reverse direction and searches the state for text copied from withheld fields. On ScienceQA it raised 33 flags, all of which we inspected and found to be false positives. The dataset’s explanation field (*solution*) begins by quoting the question, and its background field (*lecture*) is copied into the visible hint field by the dataset itself. Discarding such cases as leaks would remove valid items.

4.3 Sampling and the reordered twin

Cases are drawn deterministically under a fixed seed. Sampling is stratified by class or subject with at least one case per class, so a 500-question sample of BANKING77 still covers all 77 intents. For the five set-of-choices slices marked in Table 1, every question also has a *reordered twin*: the same question with its options shuffled, so the correct answer moves to a different position. A question and its twin form a *family* and are resampled together in the bootstrap. Figure 4 shows a family on which the commercial model’s correctness flips. The text suite contains 8,016 cases in 5,516 families.

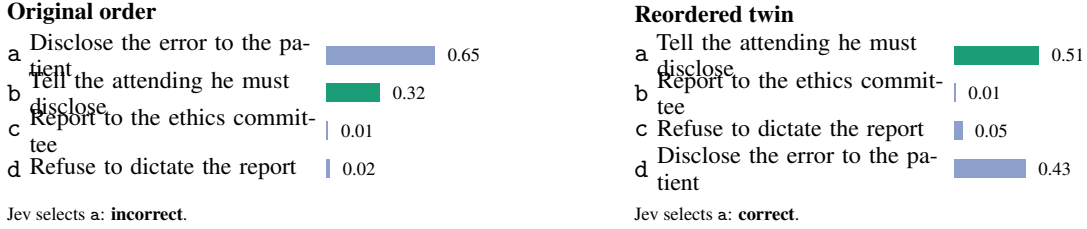


Figure 4: A reordered-twin family from MedQA-USMLE. The options and the question are identical; only their order differs. Jev 1.13 selects position a both times, which is wrong in the original order and right in the twin. Option texts are abbreviated; the correct option is drawn in green.

Table 2: Evaluated systems. Serving conditions differ and are not matched; see Section 8.

System	Description	Serving
Jev 1.13.0	Hosted commercial typed-decision model	Vendor HTTP API
djev-0.1	Open typed-decision layer over DiffusionGemma 26B-A4B via vLLM, source revision 3ce907e, one denoising step	One NVIDIA A100-SXM4-80GB, driver 580.178.04, max_model_len 8192, gpu_memory_utilization 0.85, BF16 weights and KV cache, seed 0, samples 1
Laya 421M (English)	Local non-autoregressive decision model	Apple M3 Pro, MPS, 512-token default context

4.4 Evaluation protocol

One typed question per request, so one decision per case. This simplifies scoring and makes per-decision latency comparable across slices. It also forgoes the batching regime in which this model class is reported to be fastest, which we record as a limitation in Section 8. States are capped at 12,000 characters and every truncated case is flagged; 34 ATBench cases and one jailbreak case were truncated by this cap.

5 Experimental setup

Identical case files were sent to all three. djev requests pinned seed=0 and samples=1; the hosted API exposes no equivalent control. Predictions record the decision, the full probability vector, latency, token usage and error state. They exclude the state text, so the released predictions remain publishable for sources whose text cannot be redistributed.

Cost and duration. The djev text suite completed 8,016 cases in 19.4 minutes with zero errors at concurrency 6, consuming 51 minutes of A100 time in total including model startup. The hosted API consumed 6.03M input tokens.

6 Results

6.1 Main results

Table 3 reports accuracy on every text slice. Appendix A repeats each pairwise comparison as a paired bootstrap over shared cases, which is the stronger test because all three models answered byte-identical items.

Four slices are at chance for the 421M model. MedMCQA 26.9 against 25.0, MedQA 26.1 against 25.0, MMLU-Pro 12.5 against 11.1, and PubMedQA 32.8 against 33.3, which is below chance. The same model reaches 93.0% on jailbreak classification, so its competence varies sharply across slices.

6.2 Decision threshold and calibration

Two observations follow. **The commercial model’s worst slice is a threshold effect.** Its AUROC on prompt injection is 0.978, meaning it separates the classes almost perfectly; its accuracy at 0.5 is 73.3% because its probabilities sit below the threshold. **djev-0.1 is more severely affected**, with optimal thresholds between 0.002 and 0.147 and expected

Table 3: Accuracy with 95% percentile bootstrap intervals over 2,000 resamples grouped by family. SST-5 reports mean absolute error on the expected level, where lower is better. CHANCE is the mean per-case random-guess rate, accounting for varying option counts.

Slice	n	Chance	Laya 421M	djev-0.1	Jev 1.13
jailbreak_class.	400	35.2	93.0 [90.3, 95.5]	96.5 [94.5, 98.3]	97.5 [96.0, 98.8]
scienceqa_text	1000	43.4	53.4 [50.0, 56.9]	84.7 [82.0, 87.3]	95.7 [93.9, 97.2]
atbench500	500	50.0	65.4 [61.0, 69.4]	75.8 [71.8, 79.6]	93.0 [90.8, 95.2]
medqa_usmle	1000	25.0	26.1 [22.9, 29.4]	71.2 [67.5, 74.8]	87.3 [84.4, 90.0]
aegis2	500	50.0	54.2 [49.6, 58.8]	81.0 [77.4, 84.2]	82.8 [79.4, 86.2]
banking77	1000	1.3	36.3 [32.6, 40.1]	71.4 [67.8, 75.0]	81.3 [78.0, 84.5]
aegis2_response	500	50.0	52.8 [48.6, 57.2]	69.2 [65.2, 73.0]	81.2 [77.8, 84.8]
mmlu_pro	1000	11.1	12.5 [10.2, 15.1]	42.3 [38.5, 45.9]	80.6 [77.3, 83.8]
medmcqa	1000	25.0	26.9 [23.6, 30.4]	53.8 [50.1, 57.4]	78.1 [74.5, 81.5]
prompt_injections	116	51.7	81.0 [73.3, 87.9]	64.7 [56.0, 73.3]	73.3 [64.7, 81.0]
pubmedqa	500	33.3	32.8 [28.8, 37.2]	62.2 [58.2, 66.4]	70.6 [66.6, 74.6]
sst5 (MAE ↓)	500	n/a	0.849 [0.796, 0.905]	0.544 [0.501, 0.588]	0.507 [0.464, 0.551]

Table 4: Binary slices at the conventional 0.5 threshold, at the accuracy-maximising threshold, and threshold-free. Accuracy at the best threshold is selected on the data it scores and is therefore optimistic; we report it as a diagnostic of available headroom.

Slice	Model	Acc @0.5	Best thr.	Acc at best	AUROC	ECE
prompt_injections	Jev	73.3	0.07	94.8	0.978	0.162
prompt_injections	djev	64.7	0.002	83.6	0.880	0.333
prompt_injections	Laya	81.0	0.253	85.3	0.922	0.056
aegis2_response	djev	69.2	0.005	81.0	0.879	0.267
atbench500	Jev	93.0	0.33	97.0	0.993	0.139
atbench500	djev	75.8	0.147	80.6	0.855	0.190
aegis2	djev	81.0	0.007	84.2	0.914	0.168
jailbreak_class.	Laya	93.0	0.963	99.5	0.998	0.012

calibration error up to 0.333. Laya shows the opposite sign on jailbreak classification, with an optimal threshold of 0.963.

The direction and magnitude of the offset differ by implementation and by slice, so there is no single correction. The practical consequence is that any comparison of these systems at a fixed 0.5 threshold measures probability placement as much as decision quality, and every accuracy figure in Table 3 understates the model that produced it.

6.3 Option-order sensitivity

An order of magnitude separates the commercial model from the other two. For djev and Laya this places a floor on interpretation: differences of a few points between multi-option slices are within the noise introduced by option placement alone.

For Laya the mechanism is direct. On MedMCQA it selected the first option 431 times out of 1,000 while the gold answer was in first position 267 times, and on MMLU-Pro it selected the first option 247 times against a gold frequency of 100. The commercial model’s selections tracked the gold distribution closely (274/272/236/218 against 267/260/248/225). Near-chance accuracy combined with order-driven answers indicates no measurable task competence on these slices.

6.4 Effect of context budget

Laya’s figure is exactly its context limit on every case, meaning the trajectories were internally truncated. Our own truncation flag did not catch this, because it tracks only the harness’s character cap. Part of Laya’s deficit on this slice is therefore a budget difference rather than a capability difference, and we do not attempt to attribute the split. djev at 8192 tokens saw substantially complete inputs, so its 17-point gap to the commercial model on the same slice is not explained by truncation.

Table 5: Fraction (%) of matched families whose correctness changed when options were reordered. Reordering does not change the question, so any value materially above zero is positional preference.

Slice	Jev 1.13	djev-0.1	Laya 421M
banking77	2.6	12.0	16.2
medqa_usmle	3.8	14.4	20.6
scienceqa_text	3.4	15.4	33.6
mm1u_pro	5.2	20.6	13.0
medmcqa	5.4	26.4	21.8

Table 6: The same 500 ATBench trajectories, input tokens as reported by each serving stack.

Model	Mean input tokens	Accuracy
Laya 421M	512	65.4
djev-0.1	1,988	75.8
Jev 1.13	2,253	93.0

Benchmarks comparing models with different context budgets should report per-case input tokens as measured by each serving stack. Tokenizer differences alone do not account for gaps of this size: on MedMCQA the same cases were 79 tokens for Laya against 386 for the commercial model.

6.5 Probability mass on the gold option

djev assigned non-zero mass to the correct option in every one of 4,500 multi-option cases, including all 1,000 at 77 options. This is consistent with its documented mechanism of reading the probabilities of the allowed label tokens directly rather than relying on the answer appearing in a truncated top- k list. Laya failed this on more than a third of BANKING77 cases. Any evaluation relying on the log-likelihood of the gold option must report this count, since a single zero makes the average infinite and clipping conceals it.

6.6 Image-grounded decisions

We evaluated the image path of the open implementation. Neither of the other systems could be evaluated here: the commercial API route available to us has no image input, and the 421M model is text-only.

The 1.9-point difference between the image and text rows of ScienceQA has overlapping intervals.

Verification that the image is consumed. Sending 12 VQA-RAD cases twice, identical except for the image, every pair increased input tokens by 254 to 274 with no exceptions, and 3 of 12 decisions changed. With the image removed the model answered “no” to all 12 cases, producing varied answers only when the image was present. Accuracy with the image on that small sample was lower than without (66.7% against 75.0%), but the no-image condition is a degenerate always-negative predictor and 9 of the 12 golds were negative; we report it to forestall the misreading.

Latency. Image decisions cost 1,633 ms (VQA-RAD) and 1,913 ms (ScienceQA) at the median against 781 ms for text on the same hardware.

7 Discussion

Calibration is the binding constraint. The largest single improvement available to any model in this study comes from moving the decision threshold: 21.5 points for the commercial model on prompt injection, 18.9 for djev-0.1 on Aegis responses. Since these systems are deployed as gates whose threshold determines an action, this changes which actions a deployed system takes. Threshold selection belongs on a calibration split, and vendors should publish a recommended operating point, since integrators otherwise default to 0.5.

The interface admits non-overlapping competence. A 421M model at chance on four multi-option slices reached 93.0% on jailbreak classification with an AUROC of 0.998, and beat both larger systems on prompt injection at the default threshold. Buyers of this model class should evaluate on their own decision shape, and benchmark designers should resist reporting an aggregate score.

Table 7: Count of cases where the correct option received exactly zero probability.

Slice	Jev 1.13	djev-0.1	Laya 421M
banking77 (77 options)	59	0	349
pubmedqa	27	0	0
mmlu_pro	17	0	0
medmcqa	8	0	0

Table 8: Image-grounded decisions, djev-0.1. The ScienceQA pair is a controlled comparison: identical model, adapter and dataset, differing only in whether the answer requires the image.

Slice	n	Accuracy	Notes
scienceqa_image	500	82.8 [79.6, 86.2]	macro-F1 0.825, ECE 0.086
scienceqa_text	1000	84.7 [82.0, 87.3]	same dataset, rows without images
vqa_rad	251	78.1 [72.9, 83.3]	base rate 47%, AUROC 0.884

Reading a question and choosing among options is the discriminating capability. The ordering across implementations is tightest on short binary safety decisions, where all three cluster between 93.0% and 97.5% on jailbreak classification, and widest on multi-option knowledge tasks, where the spread on MMLU-Pro is 12.5 to 80.6. Option-order sensitivity follows the same pattern.

8 Limitations

1. **The commercial system is a black box.** We reach Jev 1.13.0 only through its HTTP API. Its architecture, parameter count, tokenizer, training data and serving configuration are unpublished. The only version identifier available is the returned model string, so we cannot exclude server-side changes between our run and a replication. We also cannot determine whether any slice appears in its training data, which is why its exposure flags read unknown and not absent. Every comparison in this paper is therefore between deployed systems as a customer reaches them, and a difference we attribute to one model may come from its serving stack.
2. **The commercial training method is undocumented.** The vendor describes its training as RLCD, Reinforcement Learning for Calibrated Decisions, and states that it optimises for “epistemically honest probabilities” [4]. No paper, technical specification, objective function, reward definition or training data description has been published, and we have not reproduced the method. We therefore cannot test whether the calibration behaviour we measure follows from this objective, and we attribute no measured behaviour to a training method.
3. **Prompt wording is ours.** We wrote the instructions and criteria ourselves; no vendor supplied them. A different phrasing is a different benchmark. We release the exact adapters so the wording is inspectable.
4. **One question per request.** This model class is reported to be fastest when many questions share one state. That regime is not measured here, so our latency figures should not be read as the best these systems can do.
5. **Serving conditions are not matched.** The commercial model is remote, so its latency includes a network round trip; the open implementation is on a local A100 behind an SSH tunnel, and the 421M model is on a laptop. Latency comparisons are system comparisons.
6. **Sampling controls are not matched.** djev requests pinned seed and sample count; the hosted API exposes no equivalent, so its variance is unmeasured and single-run.
7. **Training exposure.** Five slices are flagged reported or likely. Results must not be averaged across that flag.
8. **Truncation.** 34 ATBench and one jailbreak case exceeded the harness cap. Separately, Laya internally truncated every ATBench case to 512 tokens, which our flag did not detect.
9. **Balanced slices do not estimate deployment incident rates.** A prevalence-matched evaluation is required for that claim and is not included.
10. **Best-threshold accuracy is optimistic,** chosen on the data it scores.
11. **Label polarity is assumed for two sources** whose cards do not state it. We report accuracy under both polarities; in every case the assumed direction was the higher one.
12. **Single run per model.** No seed variation is reported for the hosted model, and the open implementation was run at a single fixed seed.

13. `prompt_injections` has only 116 cases, giving intervals roughly 8 points wide. Conclusions drawn from it, including the threshold finding, are correspondingly weaker than those from the 1,000-case slices.

9 Reproducibility and artifacts

Released: the pinned source manifest with commit hashes, licences and exposure flags; the download and verification scripts; the adapters; all built cases; and 24,818 predictions with full probability vectors, latency and token usage. Also released is the serving provenance for the open implementation: the exact vLLM command line, the container log, the pinned source revision and the live capability configuration.

Predictions are keyed by case identifier and carry no source text, so they can be published for sources whose text cannot be redistributed. Licences do not merge across sources: each is released with its own licence field, and SST-5, whose card states no licence, is distributed as identifiers and content hashes with no text.

[Insert archival DOI or repository URL here once the artifact is deposited.]

Use of AI assistance

The authors used AI assistance for software engineering support, drafting, and language editing: implementation help on the evaluation harness, copy-editing, and polishing of the manuscript’s prose. All experimental design, methodological decisions, and interpretation of results are the authors’ own. Every experiment was executed by the released code, and every numerical claim in this paper was checked by the authors against the released artifacts. The authors take full responsibility for the content, including the accuracy of all results, citations, and conclusions. [Adapt to the target venue’s current policy before submission.]

A Paired bootstrap over shared cases

Table 3 gives each model its own confidence interval, computed independently. Those intervals overlap whenever two systems are close, which understates what the data can settle: every model answered byte-identical cases, so the per-case correlation between two systems is available and carries information. Two models that fail the same hard items differ by less than their separate intervals suggest.

The test below resamples whole families once and reads both models on the same resample, so each difference is measured on matched items. We report the difference Δ on matched cases, its 95% percentile interval over 2,000 resamples grouped by `family_id`, whether that interval excludes zero, and the McNemar-style split: the number of cases where exactly one of the two models was correct. For `sst5` the quantity is the negated absolute error, so a positive Δ favours the first model on every row.

Three differences do not survive the paired test. On `aegis2` the gap between `Jev` and `djev` is +0.018 with interval $[-0.014, +0.050]$; on `sst5` it is +0.037 with $[-0.001, +0.074]$; on `jailbreak_classification` it is +0.010 with $[+0.000, +0.022]$. On those three slices the two systems are indistinguishable on this data, and Section 6 should be read accordingly. The fourth non-significant row is the one that matters most for our claims: `prompt_injections`, where Laya’s apparent advantage over `Jev` is -0.078 with interval $[-0.164, +0.009]$. The paired test does not establish it. Laya’s advantage over `djev` on the same slice is established, -0.164 with $[-0.250, -0.069]$.

References

- [1] Glenn W Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.
- [2] Daniel McFadden. Conditional logit analysis of qualitative choice behavior. In Paul Zarembka, editor, *Frontiers in Econometrics*, pages 105–142. Academic Press, New York, 1974.
- [3] Peter McCullagh. Regression models for ordinal data. *Journal of the Royal Statistical Society: Series B (Methodological)*, 42(2):109–127, 1980. doi: 10.1111/j.2517-6161.1980.tb01109.x.
- [4] Diogo Almeida. Introducing system one models and `jev`. TypeSafe AI blog, <https://typesafe.ai/blog/introducing-system-one-models-and-jev>, 2026. Published 28 September 2026. Accessed 29 September 2026.

Table 9: Paired bootstrap. Δ is A minus B, where A is Jev 1.13 and B is djev-0.1. A positive value favours A.

Slice	n	Δ	95% paired CI	Excludes 0	A only / B only
aegis2	500	0.018	[-0.014, 0.05]	no	36/27
aegis2_response	500	0.12	[0.082, 0.156]	yes	81/21
atbench500	500	0.172	[0.136, 0.208]	yes	91/5
banking77	1000	0.099	[0.071, 0.128]	yes	126/27
jailbreak_classification	400	0.01	[0, 0.023]	no	5/1
medmcqa	1000	0.243	[0.201, 0.285]	yes	302/59
medqa_usmle	1000	0.161	[0.126, 0.198]	yes	197/36
mmlu_pro	1000	0.383	[0.342, 0.425]	yes	411/28
prompt_injections	116	0.086	[0.043, 0.138]	yes	10/0
pubmedqa	500	0.084	[0.042, 0.126]	yes	83/41
scienceqa_text	1000	0.11	[0.085, 0.135]	yes	126/16
sst5	500	0.037	[-0.001, 0.074]	no	n/a

Table 10: Paired bootstrap. Δ is A minus B, where A is Jev 1.13 and B is Laya 421M. A positive value favours A.

Slice	n	Δ	95% paired CI	Excludes 0	A only / B only
aegis2	500	0.286	[0.24, 0.336]	yes	168/25
aegis2_response	500	0.284	[0.236, 0.334]	yes	168/26
atbench500	500	0.276	[0.232, 0.32]	yes	151/13
banking77	1000	0.45	[0.406, 0.493]	yes	475/25
jailbreak_classification	400	0.045	[0.018, 0.075]	yes	28/10
medmcqa	1000	0.512	[0.464, 0.56]	yes	564/52
medqa_usmle	1000	0.612	[0.57, 0.658]	yes	643/31
mmlu_pro	1000	0.681	[0.639, 0.721]	yes	700/19
prompt_injections	116	-0.078	[-0.164, 0.009]	no	8/17
pubmedqa	500	0.378	[0.324, 0.434]	yes	231/42
scienceqa_text	1000	0.423	[0.386, 0.463]	yes	448/25
sst5	500	0.342	[0.285, 0.404]	yes	n/a

- [5] Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient intent detection with dual sentence encoders. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*, 2020. doi: 10.18653/v1/2020.nlp4convai-1.5.
- [6] Brandon T Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.
- [7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [8] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. PubMedQA: A dataset for biomedical research question answering. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2567–2577, 2019.
- [9] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, 2013.
- [10] Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. MedMCQA: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Proceedings of the Conference on Health, Inference, and Learning*, volume 174 of *Proceedings of Machine Learning Research*, pages 248–260. PMLR, 2022.
- [11] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *arXiv preprint arXiv:2009.13081*, 2020.
- [12] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.

Table 11: Paired bootstrap. Δ is A minus B, where A is djev-0.1 and B is Laya 421M. A positive value favours A.

Slice	n	Δ	95% paired CI	Excludes 0	A only / B only
aegis2	500	0.268	[0.218, 0.314]	yes	157/23
aegis2_response	500	0.164	[0.122, 0.21]	yes	108/26
atbench500	500	0.104	[0.052, 0.154]	yes	112/60
banking77	1000	0.351	[0.306, 0.397]	yes	407/56
jailbreak_classification	400	0.035	[0.005, 0.068]	yes	28/14
medmcqa	1000	0.269	[0.222, 0.317]	yes	382/113
medqa_usmle	1000	0.451	[0.403, 0.499]	yes	522/71
mmlu_pro	1000	0.298	[0.254, 0.34]	yes	351/53
prompt_injections	116	-0.164	[-0.25, -0.069]	yes	6/25
pubmedqa	500	0.294	[0.236, 0.35]	yes	198/51
scienceqa_text	1000	0.313	[0.27, 0.356]	yes	383/70
sst5	500	0.305	[0.233, 0.375]	yes	n/a

- [13] Pan Lu, Swaroop Mishra, Tony Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [14] Jason J Lau, Soumya Gayen, Asma Ben Abacha, and Dina Demner-Fushman. A dataset of clinically generated visual questions and answers about radiology images. *Scientific Data*, 5(1):1–10, 2018.
- [15] Yu Li, Haoyu Luo, Yuejin Xie, Yuqian Fu, Zhonghao Yang, Shuai Shao, Qihan Ren, Wanying Qu, Yanwei Fu, Yujiu Yang, Jing Shao, Xia Hu, and Dongrui Liu. ATBench: A diverse and realistic agent trajectory benchmark for safety evaluation and diagnosis. *arXiv preprint arXiv:2604.02022*, 2026. doi: 10.48550/arXiv.2604.02022.
- [16] Shaona Ghosh, Prasoon Varshney, Makesh Narsimhan Sreedhar, Aishwarya Padmakumar, Traian Rebedea, Jibin Rajan Varghese, and Christopher Parisien. AEGIS2.0: A diverse AI safety dataset and risks taxonomy for alignment of LLM guardrails. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5992–6026. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.naacl-long.306.
- [17] Tilmann Gneiting and Adrian E Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.
- [18] Mahdi Pakdaman Naeini, Gregory F Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using bayesian binning. In *AAAI Conference on Artificial Intelligence*, 2015.
- [19] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017.
- [20] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. Large language models are not robust multiple choice selectors. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.03882.
- [21] Jinquan Zheng, Jia Yuan, Jiacheng Yao, Chenyang Gu, Pujun Zheng, and Guoxiu He. Mitigating selection bias in large language models via permutation-aware GRPO. *arXiv preprint arXiv:2603.21016*, 2026.
- [22] Hiroki Tamba. Position bias is hidden behind ceiling effects: A permutation diagnostic for LLM benchmarks. *arXiv preprint arXiv:2607.20864*, 2026.